

Contents

guide Scaling Laws for Neural Language Models	1
0. 这篇论文的位置	1
1. 它到底在研究什么	2
2. 为什么这篇在 2020 年重要	2
3. 三个基本 power law	2
4. Power law 怎么读	3
5. N 和 D 要一起看	4
6. Compute-optimal: 为什么论文说要大模型早停	4
7. Critical batch size	5
8. 实验设置和证据链	5
9. 本地代码怎么对应	5
10. Kaplan 和 Chinchilla 的区别	6
11. 该怎么用这篇指导训练预算	7
12. 局限和风险	7
13. 对今天的意义	7
14. 常见误解	8
15. 闭卷检查	8
16. 用 AI agent 学这篇的正确方式	9

guide_Scaling Laws for Neural Language Models

原论文: Scaling Laws for Neural Language Models
作者: Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, Dario Amodei
年份: 2020
本地原文 PDF: [learning/scaling-infra/paper/01_scaling_laws_for_neural_language_models.pdf](#)
本地代码入口:
[learning/scaling-infra/src/scaling_laws.py](#)
[learning/scaling-infra/src/capstone_train_estimator.py](#)
[learning/scaling-infra/src/tests/test_estimator.py](#)

0. 这篇论文的位置

Scaling Laws for Neural Language Models 是大模型时代的预算语言。它试图回答一个非常工程的问题: 如果我有更多参数、更多数据、更多训练算力, loss 会怎样下降? 应该把预算花在更大模型、更长训练, 还是更多数据上?

论文的核心发现是: Transformer 语言模型的 cross-entropy loss 随参数量 N 、数据量 D 、训练计算量 C 呈稳定 power law。换句话说, 在很多数量级范围内, log-log 图上接近直线。

这篇论文的历史作用很大。它让 “scale” 从经验主义变成可以外推的工程规划工具。GPT-3 时代的很多直觉, 都来自这里。

但你读它时必须同时记住一个后续事实: 2022 年 Chinchilla 论文修正了 compute-optimal 的模型/数据配比, 指出 Kaplan 风格训练通常数据不够、模型偏大。也就是说, Kaplan 2020 是 scaling law 方法论的基础, 不是今天预训练配方的最终答案。

1. 它到底在研究什么

论文研究的是 decoder-only Transformer 在语言建模上的 loss。输入是 WebText2 数据，模型目标是自回归预测 token，评估指标是 1024-token context 上的 cross-entropy loss, 单位是 nats。

作者关心三个主尺度：

N = non-embedding parameters

D = dataset size in tokens

C = training compute

其中 N 特意排除了词表 embedding 和 position embedding。论文发现，如果把 embedding 参数算进去，不同 depth/width 的模型会显得更乱；排除 embedding 后，loss 与参数量关系更干净。

训练 compute 用近似：

$$C \approx 6NBS$$

其中 B 是 batch size, S 是 optimizer update steps。直觉上，每个 token 的 forward 大约 $2N$ FLOPs, backward 约再多两倍，总共约 $6N$ 。

这个公式后来变成大模型训练估算的常用速算式：

training FLOPs $\sim 6 * \text{parameters} * \text{training_tokens}$

2. 为什么这篇在 2020 年重要

在这篇之前，大家知道“大模型更好”，但不知道它好到什么程度，也不知道继续扩大会不会很快撞墙。Scaling Laws 的贡献是用大量实验给出平滑趋势：

- 模型参数从很小到十亿级。
- 数据量跨多个数量级。
- compute 跨多个数量级。
- depth、width、heads 等架构形状在合理范围内影响相对弱。

论文不是说架构不重要，而是说：在他们测试的 Transformer family 内，规模变量 N, D, C 比架构细节更能解释 loss。

这改变了研究和工程决策方式。以前你可能问：

我要不要换一个复杂结构？

读完这篇后你会先问：

在相同预算下，增大 N, D, C 的收益曲线是什么？

当前瓶颈是模型太小、数据太少，还是训练不够？

3. 三个基本 power law

论文给出三个单变量 scaling law。这里的 L 是 loss，不包含不可约熵项的简化写法；读论文时重点看指数和趋势。

当模型参数限制性能，而数据和训练足够时：

$$L(N) = \left(\frac{N_c}{N} \right)^{\alpha_N}$$

论文拟合:

$$\alpha_N \approx 0.076, \quad N_c \approx 8.8 \times 10^{13}$$

当数据量限制性能, 而模型足够大、训练早停时:

$$L(D) = \left(\frac{D_c}{D} \right)^{\alpha_D}$$

论文拟合:

$$\alpha_D \approx 0.095, \quad D_c \approx 5.4 \times 10^{13}$$

当 compute 限制性能, 且模型大小、数据和 batch 被优化分配时:

$$L(C_{min}) = \left(\frac{C_c^{min}}{C_{min}} \right)^{\alpha_C}$$

论文拟合:

$$\alpha_C \approx 0.050$$

这些指数都很小, 所以 scaling 有明显 diminishing returns。比如参数翻倍不会让 loss 大幅下降, 而是按一个很小的幂次缓慢下降。

4. Power law 怎么读

power law 的形式是:

$$L \propto X^{-\alpha}$$

取 log 后:

$$\log L = \text{const} - \alpha \log X$$

所以在 log-log 图上是一条斜率为 $-\alpha$ 的直线。论文中很多图的力量就来自这里: 当点跨越多个数量级仍然近似成直线, 工程上就可以外推。

但外推不是魔法。它依赖前提:

- 你还在相同数据分布附近。
- 你还在相同模型 family 附近。
- 没有新的瓶颈出现。
- 数据、训练、优化没有进入另一个 regime。

所以正确读法不是 “永远按这条线下降”, 而是 “在实验覆盖范围附近, 这条线给了有用的一阶预算模型”。

5. N 和 D 要一起看

只增加模型，不增加数据，会过拟合或收益递减。只增加数据，不增加模型，模型容量会成为瓶颈。论文提出一个联合形式描述 N, D 的关系：

$$L(N, D) = \left[\left(\frac{N_c}{N} \right)^{\alpha_N / \alpha_D} + \frac{D_c}{D} \right]^{\alpha_D}$$

这个公式读起来比看起来简单：

模型太小：

第一项大，N 限制 loss

数据太少：

第二项大，D 限制 loss

两者都够：

loss 继续沿更低的 envelope 下降

论文从这个形式推出一个当时很重要的结论：为了避免数据瓶颈，数据量可以随模型大小次线性增长，大致：

$$D \propto N^{0.74}$$

这句话后来很容易被误用。它不是今天的“最佳 token/parameter ratio”，而是在 Kaplan 2020 的实验和定义下，对 overfitting 边界的拟合。

6. Compute-optimal: 为什么论文说要大模型早停

在固定 compute budget 下，你可以选择：

小模型，训练到接近收敛

大模型，训练较少 steps，远未收敛

论文结论是：compute-efficient frontier 更偏向大模型早停。也就是，不要把小模型榨到完全收敛；用更大模型，训练到还没收敛时就停止，反而在相同 compute 下 loss 更低。

论文用 C_{min} 表示达到某个 loss 所需的最小 non-embedding compute。拟合显示，compute 增加时，最优分配大致是：

$$N_{opt} \propto C_{min}^{0.73}$$

$$B_{crit} \propto C_{min}^{0.24}$$

$$S_{min} \propto C_{min}^{0.03}$$

直觉非常强：

更多 compute 主要应该花在更大模型 N 上;
batch size 可以增加, 用来并行化;
serial steps 只需要非常慢地增加。

这就是论文摘要里 “larger models are significantly more sample-efficient” 的来源。

7. Critical batch size

论文还讨论 batch size。训练并不是 batch 越大越好。存在一个 critical batch size B_{crit} :

- batch 小于 B_{crit} 时, 增大 batch 基本不损失 compute efficiency, 并提高并行度。
- batch 大于 B_{crit} 后, 继续增大 batch 会降低 compute efficiency。

论文发现 B_{crit} 主要随 loss 变化, 而不是直接由模型大小决定。模型越好、loss 越低, critical batch size 越大。大模型训练后期可以用更大 batch 保持并行效率。

对学习者来说, 这解释了为什么大规模训练计划不只算总 FLOPs, 还要算:

能不能用足够大的 batch 吃满 GPU

batch 变大后优化效率是否下降

serial steps 是否太多导致训练时间不可接受

8. 实验设置和证据链

论文的实验对象是 WebText2 上的 autoregressive Transformer。WebText2 是 WebText 的扩展版, 用 Reddit 外链网页构建, BPE 词表大小 50257, 总量约 96GB 文本。

模型主要是 decoder-only Transformer, context length 多数实验为 1024。模型大小范围从约 768 non-embedding parameters 到 1.5B non-embedding parameters。训练使用 Adam 或 Adafactor, 学习率 schedule 包含 warmup 和 cosine decay。

证据链有几层。

第一, $L(N)$: 在足够数据和训练下, loss 随 non-embedding parameter count 平滑下降。排除 embedding 参数后, depth/width 形状差异对趋势影响变弱。

第二, $L(D)$: 固定大模型, 在不同数据子集上训练并早停, loss 随 dataset tokens 呈 power law。

第三, $L(C)$ 和 $L(C_{min})$: 固定 batch 看到 compute scaling, 再用 critical batch 调整得到更干净的 compute-optimal scaling。

第四, overfitting universality: N, D 一起变化时, overfitting penalty 主要由 $N^{0.74}/D$ 这类比例控制。

第五, transfer: 模型在其他文本分布上的 loss 与 WebText2 loss 强相关, 通常像是在 loss 上加一个近似常数 offset。也就是说, 分布转移有 penalty, 但更好的 WebText2 模型通常也更好地迁移。

第六, context position: 更大模型在 1024-token context 的后部 token 上收益更明显, 说明大模型更能利用长上下文。

这些实验一起支撑一个主张: 对这个模型 family, loss 是平滑可预测的 scale 函数。

9. 本地代码怎么对应

learning/scaling-infra/src/scaling_laws.py 同时放了 Kaplan 和 Chinchilla 的教学函数。

Kaplan 简化函数:

```
def kaplan_loss(N):
    Nc = 8.8e13
    alpha = 0.076
    return (Nc / N) ** alpha
```

它对应论文里的 $L(N)$ 单变量参数 scaling。

Chinchilla 函数:

```
def chinchilla_loss(N, D, A=406.4, B=410.7, E=1.69,
                    alpha=0.34, beta=0.28):
    return E + A * (N ** -alpha) + B * (D ** -beta)
```

这是后续 Hoffmann 2022 的形式，不属于 Kaplan 原论文。它把 loss 分成 irreducible term、model term、data term，用来说明今天更常用的 compute-optimal 训练倾向于更多数据。

训练 FLOPs 估算:

```
def compute_flops(N, D):
    return 6 * N * D
```

这里的 D 是训练 token 数。如果一个 7B 模型训练 1T tokens:

$C \approx 6 * 7e9 * 1e12 = 4.2e22$ FLOPs

capstone_train_estimator.py 把这个估算接到工程计划:

输入:

model size, seq len, batch, training tokens, GPU count, VRAM, TFLOPS

输出:

parallel strategy, memory per GPU, throughput, training hours, cost

这就是 scaling laws 对工程最直接的用法: 不是只预测 loss，而是把 loss/预算/硬件连起来。

10. Kaplan 和 Chinchilla 的区别

这部分非常重要。很多学习者会把两篇混在一起。

Kaplan 2020 的 compute-optimal 结论偏向:

更大模型
较少数据
早停

因为他们发现大模型 sample-efficient，最优 serial steps 增长很慢。

Chinchilla 2022 重新研究后指出，很多大模型其实训练 token 不够。它给出的 compute-optimal 经验更接近:

参数 N 和 token D 同时增加

D / N 大约在 20 tokens per parameter 量级

所以本仓库 scaling_laws.py 里有:

```
def chinchilla_optimal_split(C):
    ratio = 20.0
    N = (C / (6 * ratio)) ** 0.5
```

```
D = C / (6 * N)
return int(N), int(D)
```

也有 over-training split:

```
def over_train_split(C, ratio=200.0):
    N = (C / (6 * ratio)) ** 0.5
    D = N * ratio
    return int(N), int(D)
```

它用于理解 Llama 3 这类现代模型为什么会用远超 Chinchilla 20:1 的 token/parameter ratio。原因通常不是训练 loss 最优这么简单，还包括推理成本、部署复用、数据质量、长尾能力和多轮 post-training 等因素。

11. 该怎么用这篇指导训练预算

一个训练预算估算的最小流程:

1. 设定模型参数 N
2. 设定训练 token D
3. 估算 FLOPs = $6ND$
4. 根据 GPU 数和有效 TFLOPS 估算训练时间
5. 用 scaling law 粗略判断是否模型太小或数据太少
6. 用 Chinchilla/现代 recipe 修正 $N:D$ 配比
7. 再考虑显存、并行策略、checkpoint、数据加载和容错

本地 estimator 的核心吞吐估算是:

```
tok_per_s_per_gpu = mfu * gpu_tflops * 1e12 / (6 * n_params)
total_tok_s = tok_per_s_per_gpu * n_gpu
hours = n_tokens / total_tok_s / 3600
```

这个式子把模型参数越大、每 token 计算越贵这件事体现出来。即使 scaling law 说大模型 sample-efficient，硬件时间和推理成本仍然是真约束。

12. 局限和风险

第一，论文只直接研究了特定数据、特定 tokenizer、特定 Transformer family、最大约 1.5B non-embedding 参数。外推到百亿、千亿需要谨慎。

第二，loss 不是全部能力。论文也提醒，smooth loss improvement 可能掩盖能力上的 qualitative changes。反过来，loss 改善也不保证所有下游任务同步改善。

第三，compute 估算忽略了 context-dependent attention 项、通信、pipeline bubble、checkpoint overhead、数据加载等系统因素。真实训练计划必须在 $6ND$ 之外加工程折扣。

第四，Kaplan 的 compute-optimal 结论后来被 Chinchilla 修正。今天不能直接照搬“极大模型、少数据、早停”。

第五，数据质量没有被充分建模。token 数只是粗略量纲，低质量重复 token 和高质量 instruction/code/math token 的价值不同。

13. 对今天的意义

今天读这篇，重点不是背 $\alpha_N = 0.076$ ，而是学会一种思维方式:

把训练结果变成可拟合曲线；
把曲线变成预算选择；
把预算选择接到硬件和数据现实。

它让你能看懂现代 pretraining recipe 里的问题：

- 为什么要报告 training tokens？
- 为什么要报告 FLOPs？
- 为什么数据配比和去重会影响 scaling？
- 为什么 7B/13B/70B 不是随便选的尺寸？
- 为什么训练预算和推理预算可能冲突？
- 为什么现代模型会 over-train 小一些的 dense model？

这篇还会帮助你正确使用 AI agent。不要只问“帮我总结 scaling laws”，而要让 agent 帮你做预算表、画 log-log 图、检查是否混淆 Kaplan/Chinchilla。

14. 常见误解

误解一：scaling law 证明架构不重要。

不对。论文只说在他们研究的 Transformer family 和合理范围内，scale 比 shape 更能解释 loss。

误解二：Kaplan 2020 的 compute-optimal recipe 今天仍是最终答案。

不对。Chinchilla 修正了模型/数据配比，现代 Llama 3 又因为推理和数据策略进一步 over-train。

误解三：FLOPs = 6ND 就是精确训练成本。

不对。它是核心矩阵乘的近似，真实成本还受 attention context、通信、MFU、重算、数据加载、容错影响。

误解四：loss power law 可以无限外推。

不对。论文自己也讨论了 scaling law 迟早会 break down，尤其数据瓶颈和不可约损失会出现。

误解五：更大模型一定更适合部署。

不对。更大模型可能训练 compute-efficient，但推理成本更高。很多现代模型选择更多 tokens 训练较小模型，就是在平衡部署成本。

15. 闭卷检查

读完后你应该能回答：

1. 为什么论文把 embedding 参数从 N 中排除？
2. $C \approx 6NBS$ 中 6 的直觉来源是什么？
3. $L(N)$ 、 $L(D)$ 、 $L(C)$ 三个 power law 分别在什么瓶颈条件下成立？
4. 为什么 power-law 在 log-log 图上是直线？
5. 为什么只增加 N 不增加 D 会进入收益递减？
6. $D \propto N^{0.74}$ 在论文中是什么意思？
7. Kaplan 为什么得出“大模型早停”是 compute-efficient 的结论？
8. B_{crit} 解决的是并行效率还是模型质量问题？
9. Chinchilla 和 Kaplan 的 compute-optimal 配比有什么冲突？
10. 为什么 6ND 不等于真实集群训练账单？
11. 为什么 loss 改善不等于所有能力同步改善？
12. 本地 scaling_laws.py 中哪些函数属于 Kaplan，哪些属于 Chinchilla？

16. 用 AI agent 学这篇的正确方式

第一轮，让 agent 只画变量关系：

请画出 N 、 D 、 C 、 B 、 S 、 C_{min} 、 B_{crit} 的关系图。
每条边写清楚公式或直觉，不要总结论文。

第二轮，让 agent 做手算：

给定 $N=7e9$ ， $D=1e12$ ，请计算训练 FLOPs $\approx 6ND$ 。
再假设 8 张 GPU，每张有效 150 TFLOPS，估算训练小时数。
指出这个估算漏掉哪些系统开销。

第三轮，让 agent 区分 Kaplan/Chinchilla：

请用同一个 compute budget C ，分别按 Kaplan 大模型早停直觉
和 Chinchilla $D/N=20$ 的直觉给出 N, D 选择，并解释差异。

第四轮，让 agent 考你：

请问我 10 个闭卷题，专门检查我是否混淆：
参数 scaling、数据 scaling、compute scaling、训练 FLOPs、推理成本。

最后你要能闭卷说出：

Scaling Laws 让训练预算从拍脑袋变成可拟合的幂律外推；
但 Kaplan 2020 的 compute-optimal 结论必须和 Chinchilla 以及现代部署约束一起读。